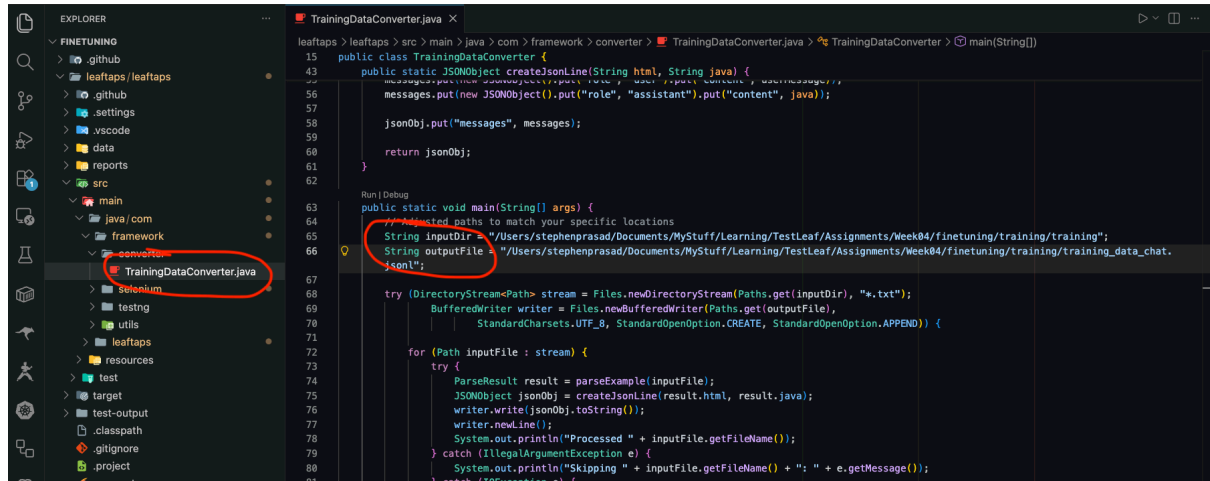
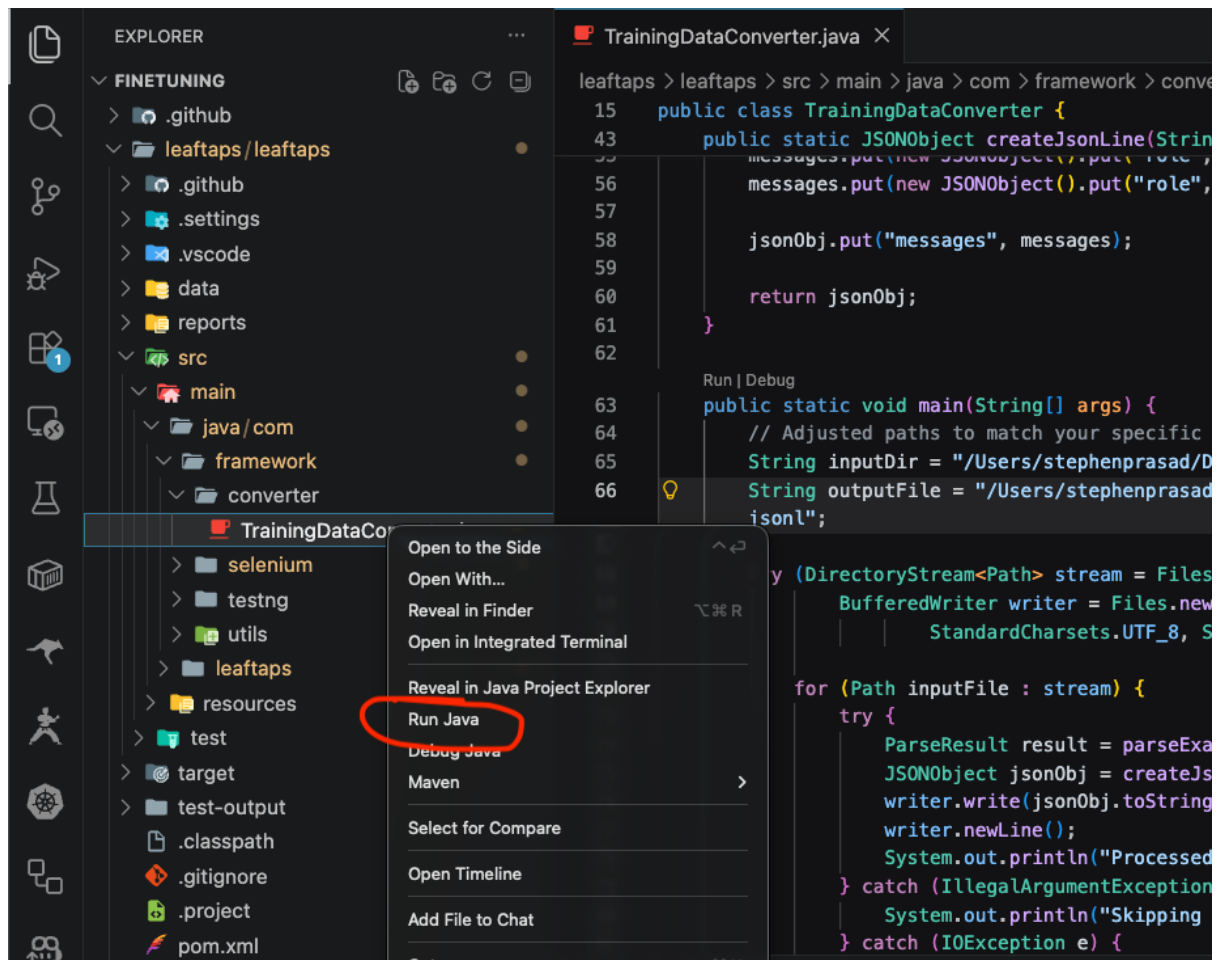


Generate JSONL file and use this for finetuning the model using testleaf API

Step 1: Open the `src/main/java/com/framework/converter/TrainingDataConverter.java` file and edit the `inputDir` and `outputFile` values.



Step 2: Right-click the file in the project window and select “Run Java” as command.



Step 3: Verify that the jsonl file has been created in the output directory specified.

```
stephenprasad@Stephens-MacBook-Pro finetuning % cd training
stephenprasad@Stephens-MacBook-Pro training % ll
total 56
drwxrwxrwx@ 12 stephenprasad  staff   384 24 Jan 11:01 training
-rw-rw-rw-@  1 stephenprasad  staff 25359 24 Jan 21:15 training_data_chat - Copy.jsonl
stephenprasad@Stephens-MacBook-Pro training % cd /Users/stephenprasad/Documents/MyStuff/Learning/TestLeaf/Assignments/week04/finetuning ; /usr/bin/en
v /Users/stephenprasad/.vscode/extensions/redhat.java-1.54.0-darwin-arm64/jre/21.0.10-macosx-aarch64/bin/java @/var/folders/hc/g1654n2565x_mynds1dwt_g
m0000gn/T/cp_616v80j1kp0uz2j1uv8k4yd6.argfile com.framework.converter.TrainingDataConverter
Processed FindLeadsUpdated.txt
Processed ViewLeadPage.txt
Processed Login.txt
Processed MyLeadsPage.txt
Processed UploadPage.txt
Processed OrderEntryHeaderPage.txt
Processed MyHomePage.txt
Processed HomePage.txt
Processed EditLeadPage.txt
Processed FindLeadsPage.txt
Conversion complete. Output saved to: /Users/stephenprasad/Documents/MyStuff/Learning/TestLeaf/Assignments/Week04/finetuning/training/training_data_ch
at.jsonl
stephenprasad@Stephens-MacBook-Pro finetuning % ll
total 0
drwxrwxrwx@ 3 stephenprasad  staff   96 23 Jan 14:24 leafaptas
drwxrwxrwx@ 5 stephenprasad  staff  160 28 Apr 20:08 training
stephenprasad@Stephens-MacBook-Pro finetuning % ll training
total 112
drwxrwxrwx@ 12 stephenprasad  staff   384 24 Jan 11:01 training
-rw-rw-rw-@  1 stephenprasad  staff 25359 24 Jan 21:15 training_data_chat - Copy.jsonl
-rw-rw-rw-@  1 stephenprasad  staff 25349 28 Apr 20:08 training_data_chat.jsonl
stephenprasad@Stephens-MacBook-Pro finetuning % []
```

Step 4: Login to testleaf api server and select “Fine-tuning” in the left menu, Click “Create” at the top right; in the modal window that opens, select “Supervised” as method, “gpt-4o-mini-2024-07-18” as Base Model and upload the generated JSONL file; click “Create Model”

Create a fine-tuned model

METHOD

Supervised

BASE MODEL

gpt-4o-mini-2024-07-18

TRAINING DATA

Upload new

Select existing

file-AcN6Dvwdc1qvYekbBmVGJb

VALIDATION DATA

Upload new

Select existing

None

SEED

Controls reproducibility. Same seed + parameters produces the same result. Leave 0 to auto-generate.

0

HYPERPARAMETERS

Batch Size

Learning Rate Multiplier

Number of Epochs

Cancel

Create Model

Step 5: Allow model to run fine-tuning – this may take 15-20 mins. Once done, verify that you see the fine-tuned model with a prefix of “ft:” in your list.

ft:gpt-4o-mini-2024-07-18:testleaf-2::DZC1PKgD	gpt-4o-mini-2024-07-18	supervised	 Succeeded	53,290	27 Apr 2026 14:12	
1-2 of 2 < 1 > 10 / page						

Step 6: Click on the “view/eye” icon at the extreme right of the fine-tuned model and verify status as succeeded

ft:gpt-4o-mini-2024-07-18:testleaf-2::DZC1PKgD

Status

 Succeeded

Job ID

ftjob-xwcrqwWgb2nx9dDrLjsWqRQ

Training Method

supervised

Base Model

gpt-4o-mini-2024-07-18

Output Model

ft:gpt-4o-mini-2024-07-18:testleaf-2::DZC1PKgD

Created At

27 Apr 2026 14:12:06

Trained Tokens

53,290

Epochs

10

Batch Size

1

LR Multiplier

1.8

Seed

1161945528

Training File

file-Waukyjg6yGmNFt35sHzne8

Validation File

-

Checkpoints

ft:gpt-4o-mini-2024-07-18:testleaf-2::DZC1PKgD

Re-run as Job

Close

Step 7: Verify Events show as succeeded and check **loss log** and **accuracy report** for graphs

ft:gpt-4o-mini-2024-07-18:testleaf-2::DZC1PKgD

ft:gpt-4o-mini-2024-07-18:testleaf-2::DZC1PF80:ckpt-step-90

ft:gpt-4o-mini-2024-07-18:testleaf-2::DZC1OoJ4:ckpt-step-80

Event Log

Loss Log

Accuracy Report

14:12:06

Validating training file: file-Waukyjg6yGmNFt35sHzne8

14:12:06

Created fine-tuning job: ftjob-xwcrqwWgb2nx9dDrLjsWqRQ

14:13:14

Files validated, moving job to queued state

14:13:16

Fine-tuning job started

14:24:23

Evaluating model against our usage policies

14:24:23

New fine-tuned model created

14:24:23

Checkpoint created at step 90

14:24:23

Checkpoint created at step 80

14:36:29

Usage policy evaluations completed, model is now enabled for sampling

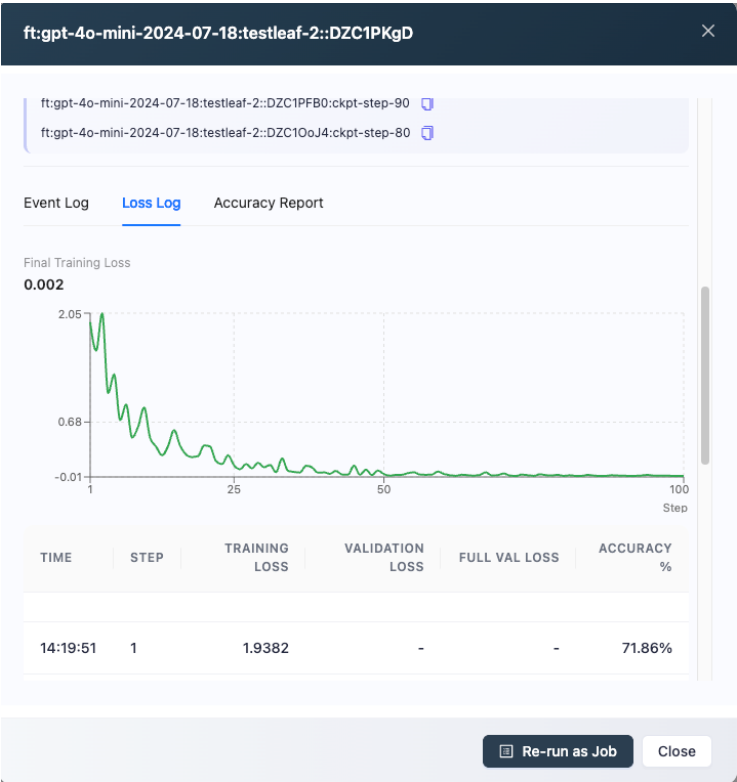
14:36:31

The job has successfully completed

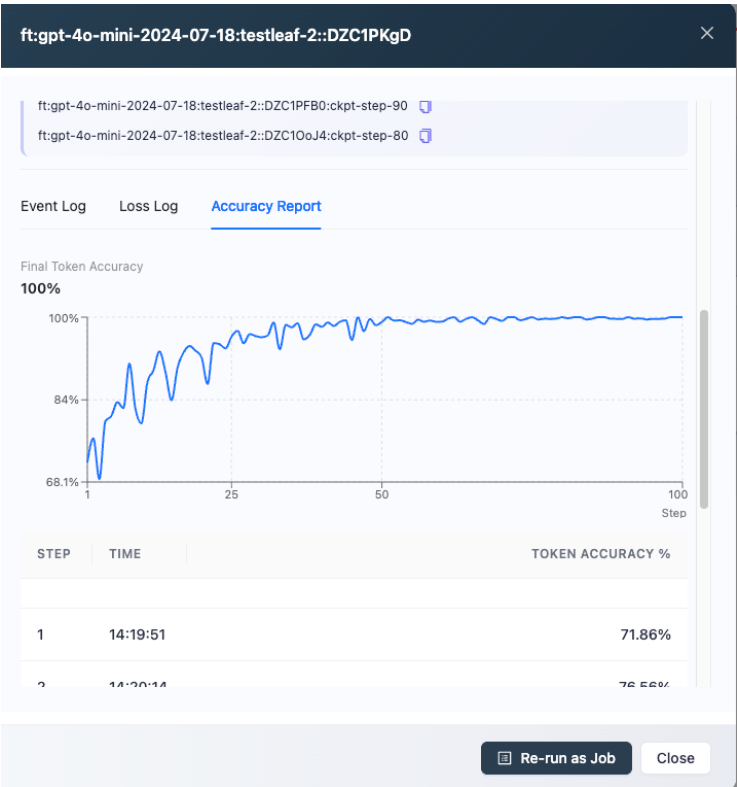
Re-run as Job

Close

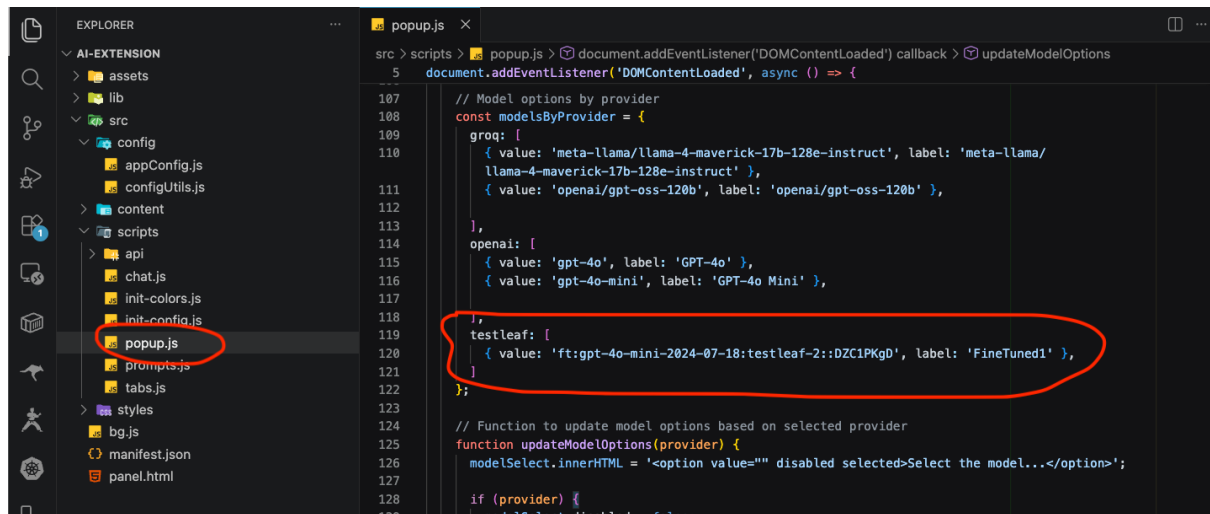
Loss log:



Accuracy Report:



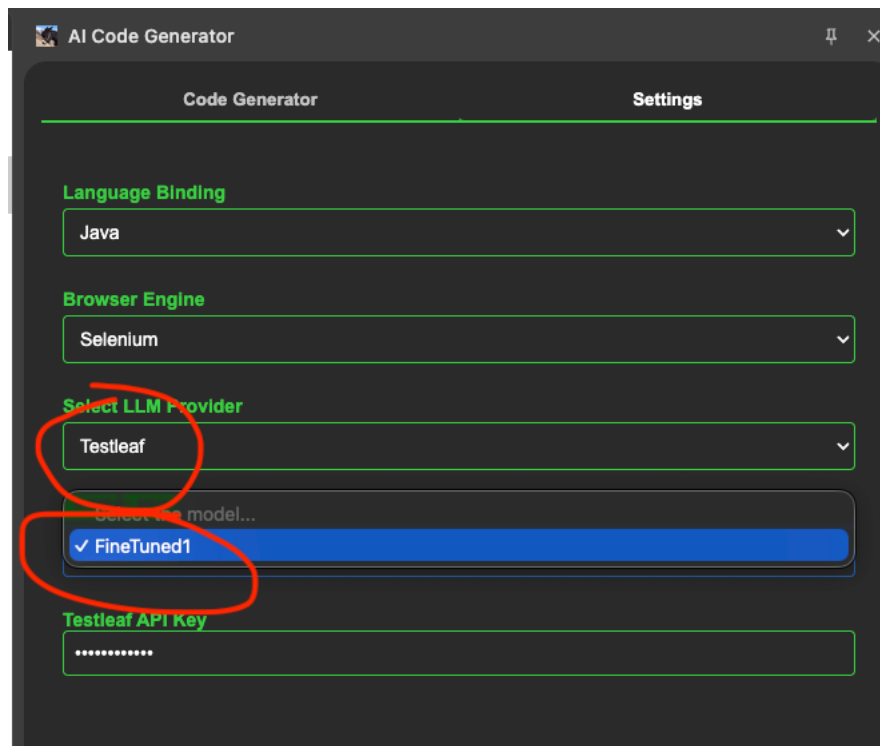
Step 8: IMPORTANT: Open the ai-extension app in vscode and navigate to popup.js file where you can check line ~108 for '*const modelsByProvider*' entry. Add a model entry for ft:gpt-xxxxxx::xxxxx with label as 'FineTuned1'



```
src > scripts > popup.js > document.addEventListener('DOMContentLoaded') callback > updateModelOptions
5 document.addEventListener('DOMContentLoaded', async () => {

107 // Model options by provider
108 const modelsByProvider = {
109   groq: [
110     { value: 'meta-llama/llama-4-maverick-17b-128e-instruct', label: 'meta-llama/llama-4-maverick-17b-128e-instruct' },
111     { value: 'openai/gpt-oss-120b', label: 'openai/gpt-oss-120b' },
112   ],
113   openai: [
114     { value: 'gpt-4o', label: 'GPT-4o' },
115     { value: 'gpt-4o-mini', label: 'GPT-4o Mini' },
116   ],
117   testleaf: [
118     { value: 'ft:gpt-4o-mini-2024-07-18:testleaf-2::DZC1PKgD', label: 'FineTuned1' },
119   ],
120 };
121
122 // Function to update model options based on selected provider
123 function updateModelOptions(provider) {
124   modelSelect.innerHTML = '<option value="" disabled selected>Select the model...</option>';
125   if (provider) {
126     modelSelect.disabled = false;
127   }
128 }
```

Step 9: FINAL: Reload the ai-extension and validate that new, finetuned model is displayed in list



Step 10: VALIDATE: Select a form and generate a page class using the fine tuned model to validate correctness

The screenshot displays the Testleaf application interface. On the left, a 'Leaflaps Login' form is highlighted with a red rectangle. The form contains fields for 'Username' and 'Password', and a 'Login' button. On the right, a chat window titled 'Reset Chat' shows a Java code snippet for a 'LoginPage' class. The code includes methods for entering username and password, and a 'clickLogin()' method. A red checkmark is visible next to the code. At the bottom of the chat window, there are 'Inspect' and 'Generate' buttons, both of which are circled in red. The footer of the application reads 'Welcome to Leaflaps Application - TestLeaf environment for automation engineering learning.'

Generated code (using finetuned model) below:

```
package com.testleaf.pages;
```

```
import com.framework.selenium.api.design.Locators;
```

```
import com.framework.testng.api.base.ProjectSpecificMethods;
```

```
/**
```

```
 * Page Object representing the Login page.
```

```
 * Provides methods to interact with the login form fields and actions.
```

```
 * @author Auto-generated
```

```
 */
```

```
public class LoginPage extends ProjectSpecificMethods {
```

```
 /**
```

```
* Enters the username in the login form.  
* @param uName The username to enter  
* @return The current LoginPage instance  
*/
```

```
public LoginPage enterUsername(String uName) {  
    clearAndType(locateElement(Locators.ID,"username"),uName);  
    reportStep(uName+" username is entered successfully", "pass");  
    return this;  
}
```

```
/**  
* Enters the password in the login form.  
* @param passWord The password to enter  
* @return The current LoginPage instance  
*/
```

```
public LoginPage enterPassword(String passWord) {  
    clearAndType(locateElement(Locators.ID,"password"),passWord);  
    reportStep(passWord+" password is entered successfully", "pass");  
    return this;  
}
```

```
/**  
* Clicks the Login button to submit the form.  
* @return A new HomePage instance  
*/
```

```
public HomePage clickLogin() {  
    click(locateElement(Locators.CLASS_NAME, "decorativeSubmit"));  
    reportStep("Login button is clicked successfully", "pass");  
}
```

```
return new HomePage();
```

```
}
```

```
}
```